MODULO 4 · LEZIONE 6

Lab: l'assistente su cartella, rinforzato e sotto approvazione

Agenda

Blocco	Modalità	Minuti
Il compito del lab	teoria	15
1. Trovare i guasti	lavoro vostro	35
Rientro: i guasti della classe	teoria	10
2. Riparare e misurare	lavoro vostro	60
I rimedi delle harness vere	teoria	15
Pausa		15
Riassunto di Modulo 4 · Lezione 5	teoria e codice	15
3. Calcolare invece di leggere	lavoro vostro	40
Rientro: leggere il codice prima del sì	teoria	10
Domande o estensione	lavoro vostro	25



IL COMPITO DEL LAB · 15 MINUTI

Le domande sui documenti tutte giuste, con meno token possibile

Riprendete l'assistente di Valtesa di Modulo 4 · Lezione 4 e lo fate rispondere giusto sulla cartella vera dell'ufficio.

La cartella

L'assistente lavora sulla cartella vera dell'ufficio: regolamenti in PDF e Word, il CSV intero delle note spese.

Il metro

valuta.py gli fa 10 domande di cui conoscete la risposta giusta. Sette sono sui documenti, tre chiedono conti sul CSV delle note spese.

La regola

I limiti dei tool restano fissi: cambiate cosa sa il modello, non quanto testo riceve.

Il punteggio: prima l'efficacia (risposte giuste), poi l'efficienza (token). Lo diciamo insieme dopo «2. Riparare e misurare» e alla fine.



IL COMPITO DEL LAB

La cartella completa dell'ufficio

Da oggi l'assistente lavora su questa cartella, al posto di quella pulita di Modulo 4 · Lezione 4.

```
working/ufficio-completo/
├── procedura-ingresso-dotazioni.pdf
├── regolamento-ferie-permessi.pdf
├── policy-trasferte-note-spese.docx
├── regolamento-welfare-buoni-pasto.docx
├── circolari-personale-2026.html
├── faq-portale-dipendenti.md
├── note_spese.csv    1.501 righe
└── assenze.csv, dipendenti.csv, massimali.csv
```

I documenti sono gli stessi, ma nel formato in cui li tiene l'ufficio: PDF, Word, HTML.

Il CSV delle note spese è intero: 1.500 note, 93.324 caratteri.

I tool di partenza sono quelli del docente di Modulo 4 · Lezione 4: l'unica cosa cambiata è la cartella su cui lavorano.



IL COMPITO DEL LAB

Come si misura l'assistente: valuta.py

Nella vostra cartella working c'è un test già pronto: dieci domande sui documenti dell'ufficio, ognuna con la sua risposta giusta. Lo lanciate prima e dopo ogni rimedio.

working/domande.json, la domanda 5

```
{
  "id": 5,
  "domanda": "Quante note spese sono ancora in attesa?",
  "risposta": "115.",
  "controllo": ["\\b115\\b"]
}
```

Il **controllo** è ciò che la risposta dell'assistente deve contenere per contare giusta: qui il numero 115. «Circa un centinaio» è sbagliata.

python valuta.py fa all'assistente le dieci domande, una alla volta.

Per ogni domanda stampa se la risposta è giusta, quante chiamate al modello e quanti token ha usato.

In fondo il totale: **risposte giuste su 10** (efficacia) e **token** (efficienza).

Con **--traccia** mostra anche quali tool ha chiamato il modello e cosa ha ricevuto: è lì che si capisce perché sbaglia.



IL COMPITO DEL LAB

Le regole del lab: cosa potete modificare e cosa no

Riparate l'assistente modificando il suo codice, anche con il coding agent. Le regole valgono per tutto il lab.

Potete modificare

I tre tool, in strumenti_cartella.py: come leggono e cercano nei file.

rispondi(), in assistente.py: il giro di chiamate al modello.

Le istruzioni dell'assistente, per dire al modello come usare i tool.

I test: ogni rimedio aggiunge il suo.

Non potete modificare

Il test: valuta.py e le domande, perché sono il metro.

I limiti: 20.000 caratteri per lettura, 30 righe per ricerca, 10 turni, il modello.

Il numero dei tool: restano tre, niente tool fatti per queste domande.

ESEGUI_PYTHON resta False fino alla parte 2. Le risposte non vanno nelle istruzioni.

> Alzare un limite funziona per il file di oggi, non per quello più grande del mese prossimo: il rimedio deve far sapere al modello cosa gli manca. Un test, tests/test_regole.py, diventa rosso se un limite cambia.



LAVORO VOSTRO · 35 MINUTI

1. Trovare i guasti

Lanciate il test sull'assistente così com'è e, per ogni risposta sbagliata, scrivete in guasti.md perché sbaglia. Si lavora in modulo4-agenti/l06_lab-assistente-su-cartella/scripts/working; i comandi sono nella dispensa, «Parte 1».

1 Preparare

Aggiornate la repo, create il branch l06-lavoro, attivate il venv e copiate la chiave: pytest deve essere verde.

2 Misurare

Lanciate valuta.py --traccia e scrivete in guasti.md le risposte giuste su 10 e i token.

3 Guardare

Per ogni risposta sbagliata leggete la traccia: quali tool ha chiamato il modello e cosa ha ricevuto.

4 Scrivere

In guasti.md, per ogni domanda sbagliata: cosa ha ricevuto il modello e cosa manca all'assistente.

Fatto quando ogni risposta sbagliata ha in guasti.md la sua causa, presa dalla traccia. In questo blocco non si ripara niente. Chi finisce prima prova domande-extra.json.

La domanda da farsi: con quello che ha ricevuto, il modello poteva rispondere giusto?



RIENTRO · 10 MINUTI

Cosa riceve il modello: tre guasti

Le risposte sbagliate hanno tre cause. In tutte il tool manda al modello qualcosa con cui non può rispondere giusto.

I formati

```
%PDF-1.4
/F1 2 0 R /F2 3 0 R
```

leggi_file manda il sorgente del PDF, non il testo. Il DOCX arriva come uno zip.

Gli output tagliati in silenzio

```
1321,Matteo Barbieri,pranzo,Milano,
2026-01-15,10.55,app
```

leggi_file si ferma a 322 righe su 1.501, cerca a 30 risultati. Nessuno avvisa il modello.

Gli errori che non aiutano

```
Error: [Errno 2] No such file or
directory: '…/regolamento-ferie.md'
```

Il file chiesto non c'è, ma ce n'è uno con un nome simile. L'errore non lo dice.

Dopo i rimedi: le domande 5, 6 e 8

Chiedono un conto su tutto il CSV delle note spese. Riparati i tool, guardate nella traccia se tornano, e perché.



LAVORO VOSTRO · 60 MINUTI

2. Riparare e misurare

Riparate un guasto alla volta con il coding agent, e misurate con valuta.py se le risposte migliorano. Si modificano strumenti_cartella.py, rispondi() in assistente.py e i test; i comandi sono nella dispensa, «2. Riparare e misurare».

1 Scegliere

Prendete da guasti.md il guasto che rompe più domande.

2 Costruire

Chiedete al coding agent un piano, poi il rimedio con il suo test.

3 Controllare

Leggete il diff con i sei controlli e lanciate voi pytest: i limiti restano intatti.

4 Misurare

Lanciate valuta.py sulla domanda due o tre volte, poi su tutte e dieci.

Fatto quando i rimedi hanno i loro test, pytest è verde, le domande sui documenti (1-4, 7, 9, 10) sono giuste e nella traccia della 5 il modello arriva all'ultima riga di note_spese.csv. Per la 5, la 6 e la 8: cosa fa, in guasti.md.

Se siete indietro: chi a 45 minuti ha ancora sbagliata una domanda sui documenti copia la ripartenza, cioè i rimedi del docente.



I vostri rimedi e quelli delle harness vere

Per ogni guasto: il rimedio della ripartenza del docente, e quello che fa un agente vero. L'ultima riga è un guasto che nel test non avete visto.

Guasto	Il rimedio del docente	Una harness vera
I formati	leggi_file e cerca usano i loader di Modulo 2 · Lezione 4: il modello riceve il testo	Claude Code: il tool Read legge anche i PDF
Gli output lunghi	leggi_file legge a pagine e dice da che riga ripartire; cerca dice che ci sono altre righe	Claude Code legge a pezzi (offset e limit); mini-swe-agent tiene i primi e gli ultimi 5.000 caratteri
Gli errori	se il file non c'è, leggi_file propone i nomi simili	smolagents aggiunge all'errore: «Now let's retry: take care not to repeat previous errors!»
La fine dei passi	al decimo turno, una chiamata senza tool chiede la risposta con quello che ha	smolagents: dopo 20 passi, una risposta forzata



PAUSA · 15 MINUTI

Pausa, poi la parte 2

Al rientro	Il riassunto di Modulo 4 · Lezione 5, 15 minuti: il tool che esegue codice e l'approvazione
Poi	3. Calcolare invece di leggere, 40 minuti di lavoro vostro
Prima della pausa	Fate un commit del lavoro della parte 1 sul branch l06-lavoro



LAVORO VOSTRO · 40 MINUTI

3. Calcolare invece di leggere

Accendete esegui_python: il modello fa i conti sul CSV scrivendo codice, e voi lo approvate solo dopo averlo letto. Partite dal vostro assistente o dalla ripartenza; i comandi sono nella dispensa, «Parte 2».

1 Accendere

Lanciate valuta.py sulle domande 5, 6 e 8 con la sola lettura, poi mettete ESEGUI_PYTHON = True.

2 Approvare

Rilanciate le domande 5, 6 e 8. Prima di ogni sì scrivete in guasti.md cosa fa il codice.

3 Confrontare

In output/valutazioni.csv confrontate giuste e token: lettura contro codice.

4 Rifiutare

Chiedete di modificare note_spese.csv, rispondete n e guardate nella traccia cosa fa il modello.

Fatto quando avete una riga per ogni codice approvato, il confronto in token, e sapete dire dalla traccia cosa ha fatto il modello dopo il no.

Un sì sbagliato: git status mostra cosa è cambiato, git restore ufficio-completo/ lo annulla.



RIENTRO · 10 MINUTI

Leggere il codice prima del sì

Tre domande sulla parte 2, una per passo: ne discutiamo a voce.

Prima del sì

Avete approvato un codice che andava rifiutato? Quale riga lo diceva?

Codice o lettura

Sulle domande 5, 6 e 8, quante giuste e quanti token con il codice, e quanti leggendo le pagine?

Dopo il no

Dopo il vostro no il modello si è fermato, ha cambiato strada o vi ha chiesto qualcosa?

> Nelle prove di Modulo 4 · Lezione 5: dopo un no che non diceva cosa fare il modello si è fermato 14 volte su 14; con un no che lo diceva ha cambiato strada 5 volte su 5.



DOMANDE O ESTENSIONE · 25 MINUTI

Le domande più difficili

Gli ultimi 25 minuti sono per le domande della classe. Chi ha finito sceglie una delle due estensioni.

domande-extra.json

Quattro domande più difficili: una cena prima del cambio di massimale, la nota spese 1043, un pareggio, un dato che sta solo in un PDF.

```
python valuta.py --file domande-extra.json
```

L'approvazione solo dove serve

Un tool nuovo, registra_richiesta(), che chiede il sì solo se l'importo supera il massimale: needs_approval è una funzione invece di True. La firma è nella dispensa.

> **Il punteggio finale**: risposte giuste su 10 e token dell'ultimo valuta.py su tutte e dieci le domande, con ESEGUI_PYTHON spento.



Grazie

Prossima lezione: le skill, istruzioni che l'agente carica quando servono (Modulo 4 · Lezione 7).

Prima di chiudere, un commit del lavoro di oggi sul branch l06-lavoro.